

Aula 12 - SQL: DDL - Criação de Bancos de Dados, Tabelas e Constraints

Descrição: Utilização da Linguagem de Definição de Dados (DDL) para materializar a modelagem relacional. Vamos aprender a criar bancos de dados (`CREATE DATABASE`), construir tabelas estruturadas (`CREATE TABLE`), definir domínios (tipos de dados) e aplicar regras de negócio fundamentais por meio de restrições de integridade.

Cenário Real:

Em 2021, uma grande plataforma de e-commerce brasileira enfrentou um apagão de pedidos que durou cerca de 40 minutos. O problema não foi a infraestrutura de rede, mas uma falha na arquitetura dos dados recém-implantada: informações sobre métodos de pagamento estavam entrando com formatos inconsistentes e clientes órfãos (sem cadastro prévio) conseguiam finalizar compras no sistema. A equipe de engenharia descobriu que, durante a migração para o novo banco de dados, a linguagem que define as estruturas físicas esqueceu de implementar regras rígidas de restrição (**constraints**) e definições adequadas de tipos de dados. Milhares de registros corrompidos tiveram que ser higienizados manualmente, gerando um prejuízo milionário. O que define a robustez de um sistema começa antes da inserção dos dados: começa na sua fundação estrutural.

O Alicerce dos Dados: Introdução à DDL e CREATE DATABASE

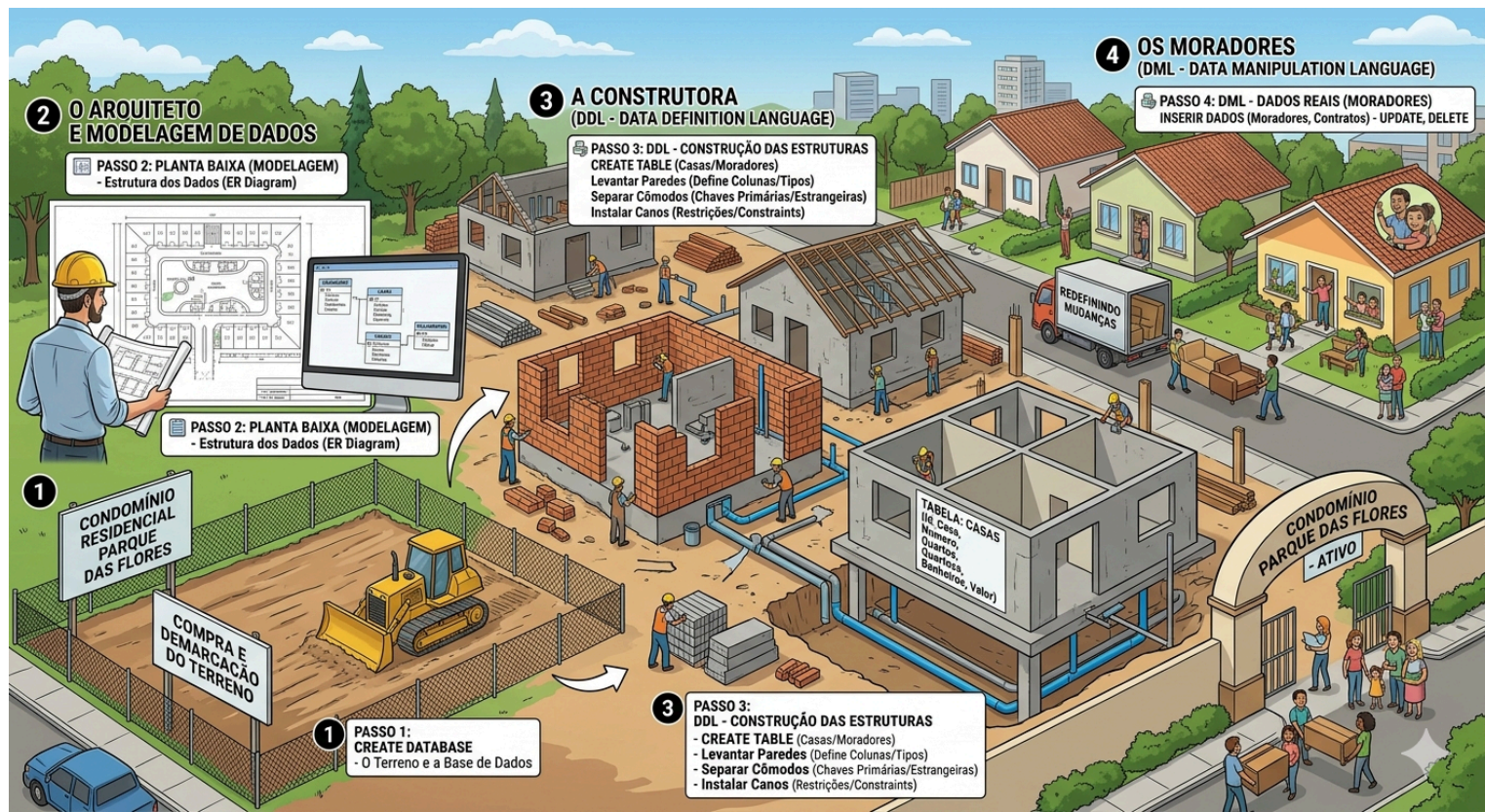
A. CONCEITO

A DDL (**Data Definition Language**) é o subconjunto da linguagem SQL responsável por projetar a infraestrutura do banco de dados. Diferente da DML, que manipula a informação (insere, atualiza, deleta), a DDL não olha para o conteúdo, mas sim para o "**recipiente**". É por meio da DDL que criamos as tabelas, índices e visões, traduzindo o modelo Entidade-Relacionamento (que fizemos no papel) para o SGBD.

O comando inicial de qualquer projeto de software é o `CREATE DATABASE`. Ele diz ao motor do banco de dados para alocar espaço em disco e criar o dicionário de dados primário. Resolver esse passo de forma otimizada permite separar diferentes aplicações no mesmo servidor físico, mantendo o isolamento e a segurança do projeto.

B. VISUALIZANDO

Imagine a construção de um condomínio residencial. O Arquiteto faz a planta baixa (Modelagem de Dados). A DDL é a construtora que levanta as paredes, separa os cômodos e instala os canos. O comando `CREATE DATABASE` é a compra e a demarcação do terreno. O comando `CREATE TABLE` (que veremos a seguir) é a construção das casas individuais. A DML (dados reais) serão os moradores que entrarão depois.



C. EXEMPLO DE CÓDIGO

SQL

-- Criando o banco de dados para o sistema de gestão do projeto

```
CREATE DATABASE db_sa;
```

-- Garantindo que vamos operar dentro do banco recém-criado

```
USE db_sa;
```

D. EXPLICAÇÃO DO CÓDIGO

- `CREATE DATABASE db_sa;`: Esta linha aciona o compilador DDL do SGBD para reservar um espaço lógico chamado `db_sa`. Se este comando falhar, nenhuma tabela poderá ser criada.
- `USE db_sa;`: Este comando "aponta" nosso cursor para o terreno correto. Sem ele, o SGBD não sabe em qual banco as próximas tabelas devem ser construídas, podendo gerar erros de "No database selected".

E. ATENÇÃO

Um erro comum de desenvolvedores é esquecer de rodar o comando `USE` após o `CREATE DATABASE`, acabando por criar tabelas no banco de sistema padrão (como o `master` no SQL Server ou `mysql` no MySQL), poluindo o ambiente de configuração do servidor.

Levantando as Paredes: CREATE TABLE e Tipos de Dados

A. CONCEITO

Uma vez dentro do banco de dados, usamos o comando `CREATE TABLE` para construir as entidades. Cada tabela representa um assunto único do mundo real (ex: Clientes, Produtos). Durante a criação da tabela, precisamos definir as **colunas** (atributos) e o **tipo de dado** (Domínio) que cada coluna aceitará.

Os tipos de dados informam ao SGBD exatamente o que esperar: números inteiros (INT), textos longos ou curtos (VARCHAR), casas decimais (DECIMAL) ou datas (DATE). Escolher o tipo correto é fundamental para otimizar o consumo de memória, melhorar a performance das buscas e evitar que o usuário digite "abacaxi" em um campo de preço.

B. VISUALIZANDO

Continuando com a brilhante analogia do condomínio: se o CREATE TABLE (DDL) é a construção da casa, os Tipos de Dados definem as regras e o formato de cada cômodo (colunas da tabela). Eles determinam exatamente que tipo de "móvel ou morador" (dados) pode entrar ali: se é um cômodo feito só para guardar números, textos longos, ou datas.

Tecnicamente, chamamos de Tipos de Dados SQL, e eles são obrigatórios na hora de usar a DDL para criar a estrutura do banco. Aqui está a tabela com os principais tipos utilizados na modelagem de banco de dados relacionais:

Tipos Numéricos

Tipo de Dado	Descrição	Exemplo de Uso
INT (ou INTEGER)	Armazena números inteiros (sem casas decimais).	Idade (Ex: 35), Quantidade de Quartos (Ex: 3).
DECIMAL(p,s) / NUMERIC	Números exatos com casas decimais. O p é o total de dígitos e s é a quantidade de casas após a vírgula.	Preço da casa, Saldo (Ex: DECIMAL(10,2) para 250000.50).
FLOAT / DOUBLE	Números com casas decimais (ponto flutuante), usados para cálculos aproximados ou científicos.	Medições, coordenadas geográficas (Ex: -26.3044).

Tipos de Texto (Strings)

Tipo de Dado	Descrição	Exemplo de Uso
VARCHAR(n)	Texto de tamanho variável. Ocupa apenas o espaço do texto inserido, até o limite máximo de n caracteres. É o mais comum.	Nome do morador (Ex: VARCHAR(100) para 'Maria Silva').
CHAR(n)	Texto de tamanho fixo. Se você inserir um texto menor que n, o banco preenche o resto com espaços em branco.	Sigla de Estado (Ex: CHAR(2) para 'SC'), CPF, CEP.
TEXT	Usado para armazenar textos muito longos, onde você não sabe o limite exato de caracteres.	Observações, histórico do cliente, descrições longas.

Tipos de Data e Hora

Tipo de Dado	Descrição	Exemplo de Uso
DATE	Armazena apenas a data (Ano, Mês e Dia).	Data de nascimento (Ex: '1990-08-15').
TIME	Armazena apenas a hora (Horas, Minutos, Segundos).	Horário de entrada (Ex: '08:00:00').
TIMESTAMP / DATETIME	Armazena a data e a hora juntas.	Data e hora exata de uma compra ou assinatura de contrato.

Tipos Lógicos (Booleanos)

Tipo de Dado	Descrição	Exemplo de Uso
BOOLEAN (ou BOOL)	Armazena valores lógicos: Verdadeiro (TRUE) ou Falso (FALSE). <i>Nota: Em alguns bancos como o SQL Server, usa-se o tipo BIT (1 ou 0).</i>	Inadimplente? (TRUE ou FALSE), Ativo? (1 ou 0).

Dica do Construtor: Escolher o tipo de dado correto na DDL é como planejar bem a fundação da casa. Usar um VARCHAR(255) para guardar a sigla de um Estado (CHAR(2)) é como construir uma garagem para caminhões quando você só tem uma bicicleta — funciona, mas desperdiça muito espaço!

C. EXEMPLO DE CÓDIGO

```
SQL
-- Criando a tabela de desenvolvedores do sistema
CREATE TABLE Desenvolvedor (
  id_dev INT,
  nome_completo VARCHAR(100),
  data_contratacao DATE,
  salario_base DECIMAL(10, 2)
);
```

D. EXPLICAÇÃO DO CÓDIGO

- CREATE TABLE Desenvolvedor: Define o nome da nossa entidade. Usamos o singular como boa prática.
- id_dev INT: Cria uma coluna de identificação numérica para cada dev.
- nome_completo VARCHAR(100): Limita o nome a 100 caracteres. Textos maiores serão truncados ou recusados, economizando memória.
- salario_base DECIMAL(10, 2): Permite salários com até 10 dígitos totais, sendo 2 reservados para os centavos (ex: 99999999.99).

E. DICA

Ao desenvolver com IAs Generativas, elas frequentemente sugerem tipos genéricos como TEXT ou FLOAT. Profissionais de banco de dados evitam FLOAT para valores monetários (pois causa arredondamentos incorretos) e sempre utilizam DECIMAL ou NUMERIC.

Regras Inegociáveis: Restrições de Integridade (Constraints)

A. CONCEITO

No mundo real, não basta apenas definir o tipo de dado; precisamos estabelecer regras de negócio rígidas. É aqui que entram as **Constraints** (Restrições). Elas são os "seguranças" do nosso banco de dados. Se uma aplicação tentar inserir um dado que viole uma constraint, o SGBD barra a operação imediatamente e devolve um erro. Isso resolve o problema de dados inconsistentes gerados por falhas no código da aplicação (front-end ou back-end).

As constraints mais vitais que utilizamos são:

- PRIMARY KEY (PK):** Garante que o registro seja único e não nulo. É a "identidade" daquela linha.

- **FOREIGN KEY (FK):** Garante a integridade referencial. Uma coluna só pode receber um valor se esse valor já existir como Chave Primária em outra tabela (ex: não posso criar uma tarefa para um projeto que não existe).
- **NOT NULL:** Obriga que a coluna sempre seja preenchida.
- **UNIQUE:** Garante que não haverá valores repetidos naquela coluna (ex: CPF, E-mail).
- **CHECK:** Valida uma condição matemática ou lógica (ex: salário deve ser > 0).

B. EXEMPLO DE CÓDIGO

SQL

-- Criação da tabela Projetos (Lado "Um" do relacionamento)

```
CREATE TABLE Projeto (  
    id_projeto INT PRIMARY KEY,  
    nome_projeto VARCHAR(100) NOT NULL,  
    orcamento DECIMAL(10, 2) CHECK (orcamento >= 0)  
);
```

-- Criação da tabela Tarefas (Lado "Muitos", que depende de Projetos)

```
CREATE TABLE Tarefa (  
    id_tarefa INT PRIMARY KEY,  
    descricao VARCHAR(255) NOT NULL,  
    status VARCHAR(20) DEFAULT 'Pendente',  
    fk_projeto INT,
```

-- Definindo a Chave Estrangeira (Relacionamento)

```
CONSTRAINT fk_tarefa_projeto  
    FOREIGN KEY (fk_projeto)  
    REFERENCES Projeto(id_projeto)  
);
```

C. EXPLICAÇÃO DO CÓDIGO

- Na tabela `Projeto`, o `id_projeto` recebe `PRIMARY KEY`, tornando-se o identificador oficial. O `nome_projeto` é `NOT NULL` (todo projeto precisa ter nome) e o `orcamento` possui um `CHECK` impedindo valores negativos.
- Na tabela `Tarefa`, usamos `DEFAULT 'Pendente'` para que, se a aplicação não enviar um status, o banco preencha automaticamente.
- A instrução `CONSTRAINT fk_tarefa_projeto FOREIGN KEY (fk_projeto) REFERENCES Projeto(id_projeto)` é o coração do modelo relacional. Ela diz: "A coluna `fk_projeto` desta tabela só aceitará números que existam na coluna `id_projeto` da tabela `Projeto`". A nomeação explícita (`fk_tarefa_projeto`) facilita a manutenção futura.

D. ATENÇÃO

A ordem de criação importa muito! Como a tabela `Tarefa` faz referência à tabela `Projeto`, você **obrigatoriamente** precisa executar o `CREATE TABLE Projeto` primeiro. Se tentar criar a tabela "filha" antes da tabela "mãe", o SGBD retornará um erro de referência inexistente. O mesmo vale na hora de deletar (`DROP TABLE`): apague as filhas antes das mães.

FAQ - Perguntas Frequentes

Pergunta: O que acontece se eu tentar apagar (DROP) a tabela Departamento agora que Funcionario depende dela?

Resposta: O SGBD vai bloquear a operação e lançar um erro de violação de chave estrangeira (Foreign Key constraint fails). Para apagar a tabela Departamento, você precisaria apagar Funcionario primeiro, ou remover a constraint que as liga.

Pergunta: Qual a diferença entre os tipos VARCHAR e CHAR? Quando usar qual?

Resposta: O CHAR(10) sempre ocupará 10 espaços no disco, mesmo se você digitar apenas "A". O VARCHAR(10) ocupará apenas o espaço necessário para o texto digitado + 1 byte de controle. Use CHAR para dados de tamanho fixo garantido (como UF do estado, ex: 'SP', 'SC') para ganhar performance, e VARCHAR para todo o resto.

Pergunta: Posso ter mais de uma PRIMARY KEY na mesma tabela?

Resposta: Não. Uma tabela só pode ter uma única Chave Primária. No entanto, essa chave primária pode ser composta por mais de uma coluna (ex: PRIMARY KEY (id_pedido, id_produto) em uma tabela de itens do pedido).

Pergunta: Esqueci de colocar uma constraint na hora de criar a tabela. Preciso dar DROP e fazer tudo de novo?

Resposta: Não. Você pode usar o comando ALTER TABLE nome_da_tabela ADD CONSTRAINT... para adicionar regras a uma tabela já existente. Profissionais fazem muito isso em sistemas em produção.

Pergunta: Posso usar IA (como o ChatGPT) para gerar meus scripts DDL?

Resposta: Sim, ferramentas de IA Generativa são ótimas para agilizar a escrita de scripts repetitivos. Porém, a IA não conhece o domínio do seu negócio. Ela pode sugerir tipos de dados ineficientes ou esquecer regras de CHECK específicas da sua empresa. Use como assistente para digitar rápido, mas o arquiteto validador da estrutura é você.

RESUMINDO

- **DDL (Data Definition Language):** Subconjunto do SQL focado na construção da infraestrutura do banco de dados (o "recipiente").
 - **CREATE DATABASE:** Instancia o espaço lógico do banco de dados no servidor. Deve ser seguido do comando USE.
 - **CREATE TABLE:** Define uma nova entidade/tabela. Exige o planejamento cuidadoso das colunas e seus domínios.
 - **Tipos de Dados:** INT, VARCHAR, DECIMAL, DATE. Escolhas críticas que afetam performance, validação de inputs e armazenamento.
 - **PRIMARY KEY:** A identificação única e irrenunciável de cada registro da tabela.
 - **FOREIGN KEY:** A restrição que garante os relacionamentos. Impede dados órfãos e exige ordem correta de criação das tabelas.
 - **Constraints (Regras de Negócio):** NOT NULL, UNIQUE, CHECK. Movem a responsabilidade pela integridade dos dados da aplicação para a fundação do SGBD.
-

MÃO NA MASSA

Contexto: Sua equipe foi contratada para desenvolver o banco de dados principal de um **Sistema de Gestão de Tarefas Corporativas**. O líder técnico repassou o diagrama de Entidade-Relacionamento e agora cabe a nós traduzir isso em código SQL (DDL). **O foco aqui é trabalhar em dupla:** a comunicação clara com seu colega é essencial para evitar erros de sintaxe e concordância dos nomes das tabelas.

Enunciado: Vamos construir a estrutura inicial do banco, criando o banco de dados, a entidade de Departamentos e a entidade de Funcionários, garantindo a integridade referencial entre elas.

Tutorial Passo a Passo:

- Passo 1: Fundação do Sistema

Crie o banco de dados (`db_gestao_corp`) e coloque-o em uso.

Verificação: Olhe na aba "Schemas" do MySQL Workbench (ou equivalente) e veja se o banco `db_gestao_corp` aparece em negrito.

- Passo 2: Criando a Tabela Independente (Domínio)

Vamos criar a tabela de Departamentos primeiro, pois ela não depende de ninguém. Utilize os seguintes dados:

- Tabela: Departamento
- Atributos: `id_depto` (PK) e `nome_depto`

- Passo 3: Criando a Tabela Dependente com Relacionamento

Agora, criamos a tabela de Funcionários. Cada funcionário deve pertencer a um departamento válido.

Utilize os dados:

- Tabela: Funcionario
- Atributos: `matricula` (PK), `nome`, `data_admissao`, `salario` (Checando o salário mínimo) e `fk_depto`.
- Restrições: `fk_func_depto`
- Chave Estrangeira: `fk_depto`
- Referência: Departamento(`id_depto`)

- Passo 4: Validação Estrutural

Execute o comando abaixo para visualizar se a tabela foi criada com a estrutura correta.

SQL

```
DESCRIBE Funcionario;
```

Resultado Esperado: Ao final, você terá o banco `db_gestao_corp` instanciado, contendo duas tabelas interligadas. Ao rodar o `DESCRIBE`, ele deve conseguir identificar claramente as colunas marcadas como PRI (Primary) e MUL (Foreign Key), confirmando que a arquitetura está pronta para receber dados reais com segurança.

FECHAMENTO

Conexão com a SA: Independentemente de qual seja o tema do projeto da sua equipe — seja um sistema de saúde escolar, um controle de ponto ou uma plataforma de adoção de animais — o conteúdo de hoje é o **alicerce** técnico do software de vocês. As tabelas que vocês acabaram de modelar no "Mão na Massa" serão as mesmas tabelas que o back-end tentará se conectar nas próximas fases do curso. Se a DDL falhar, a aplicação inteira desmorona. Garantir a criação das **Constraints** agora significa menos bugs e madrugadas em claro no dia da entrega final da SA.

Próxima Aula: Hoje nós construímos a estrutura, levantamos as paredes do prédio e colocamos portas com fechaduras seguras. Porém, nosso prédio está vazio. Na próxima aula (**Aula 13**), vamos dar vida a este ambiente utilizando a **DML (Data Manipulation Language)**. Vamos aprender a povoar as tabelas (**INSERT**), atualizar informações erradas (**UPDATE**) e limpar o que não serve mais (**DELETE**).

Antes de virem para a aula, pensem no seguinte: *Se a DDL protegeu os dados para que apenas informações válidas entrem... o que acontece se o próprio Desenvolvedor rodar um comando de alteração de dados sem filtro, afetando o banco inteiro de uma vez só?* Preparem-se para conhecer o maior pesadelo dos devs desatentos.